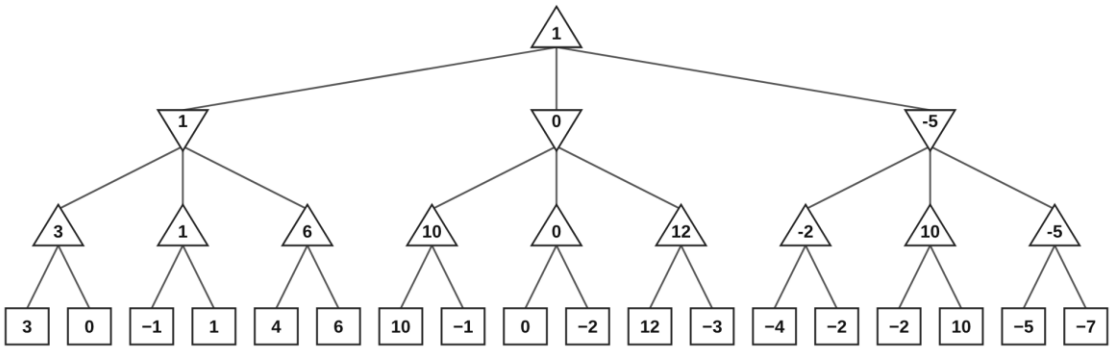
BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Phạm Đức Lương
MSSV / Lớp	3124720042 / DNT1241

Quy ước chung: Các nút con được xét từ trái sang phải. Trong hình, α và β ghi dưới dạng [α, β] tại thời điểm *vào* nút; số trong nút là giá trị nút trả về; X đánh dấu nhánh bị cắt tỉa (nút và lá bị cắt được tô xám).

Bài 1. Minimax và Alpha-Beta Pruning

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian.



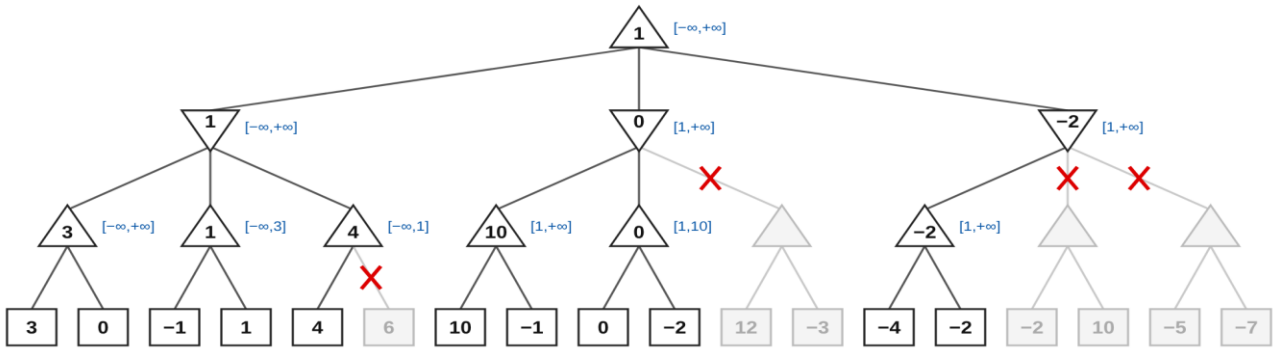
Hình 1a. Giá trị Minimax của mọi nút (gốc MAX, tầng 1 MIN, tầng 2 MAX).

Tầng MAX (mỗi nút có 2 lá), theo thứ tự trái sang phải: 3, 1, 6 | 10, 0, 12 | -2, 10, -5.
Tầng MIN: min(3,1,6) = 1; min(10,0,12) = 0; min(-2,10,-5) = -5.
Gốc MAX: max(1, 0, -5) = 1.

b) MAX nên chọn nhánh nào từ gốc? Giá trị trò chơi?

MAX chọn nhánh trái (nút MIN thứ nhất, giá trị 1). Giá trị của trò chơi là 1.

c) Áp dụng Alpha-Beta theo thứ tự trái sang phải.



Hình 1c. Alpha-Beta: [α, β] khi vào nút, giá trị trả về trong nút, X = nhánh bị cắt.

Nút	[α, β] khi vào	Diễn biến

Nút	[α, β] khi vào	Diễn biến
Gốc MAX	[-∞, +∞]	$MIN_1 \to 1$ nên α = 1; $MIN_2 \to 0$ và $MIN_3 \to \le -2$ đều không vượt 1. Kết quả = 1.
MIN_1	[-∞, +∞]	$MAX_{1a} = 3$ (β = 3), $MAX_{1\beta} = 1$ (β = 1), $MAX_{1c} = 4$. Giá trị $MIN_1 = 1$.
MAX_{1a}, $MAX_{1\beta}$	[-∞, +∞], [-∞, 3]	Duyệt hết lá (3, 0) và (-1, 1) vì chưa có nhánh nào đạt v ≥ β.
MAX_{1c}	[-∞, 1]	Lá 4 cho v = 4 ≥ β = 1 → cắt lá 6.
MIN_2	[1, +∞]	$MAX_{2a} = 10$ (β = 10); $MAX_{2\beta} = 0 \le \alpha = 1$ → cắt MAX_{2c} (lá 12, -3). $MIN_2 \le 0$.
MAX_{2a}, $MAX_{2\beta}$	[1, +∞], [1, 10]	(10, -1) → 10; (0, -2) → 0.
MIN_3	[1, +∞]	$MAX_{3a} = -2 \le \alpha = 1$ → cắt $MAX_{3\beta}$ và MAX_{3c} (lá -2, 10, -5, -7).
MAX_{3a}	[1, +∞]	(-4, -2) → -2.

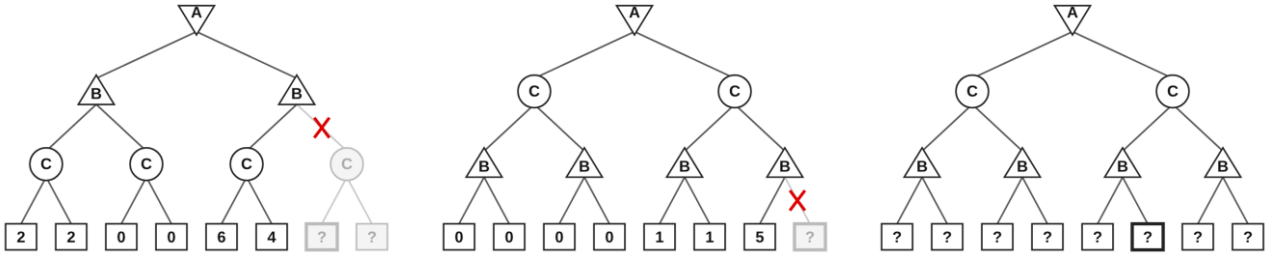
Giá trị gốc vẫn là 1, giống Minimax.

d) Liệt kê các lá không cần thăm; so sánh số lá được đánh giá.

Các lá bị cắt: 6; 12, -3; -2, 10, -5, -7 - tổng cộng 7 lá.
Minimax đánh giá cả 18 lá; Alpha-Beta chỉ đánh giá 11 lá (3, 0, -1, 1, 4, 10, -1, 0, -2, -4, -2), tiết kiệm 7/18 ≈ 39% mà vẫn cho cùng kết quả (1).

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

A là nút MIN, B là nút MAX, C là nút ngẫu nhiên (hai con xác suất ½ - ½). Không cắt tỉa khi bằng nhau, nên mọi điều kiện cắt là bất đẳng thức chặt. Nút viền đậm cần cắt được đánh dấu X.



Trường hợp 1 Trường hợp 2 Trường hợp 3

Hình 2. Ví dụ gán giá trị lá (dấu "?" = giá trị tuỳ ý, không cần biết vì đã bị cắt).

a) Trường hợp 1 (A MIN → B MAX → C ngẫu nhiên → lá; nút viền đậm là lá thứ 7, con trái của C_4).

Có thể. Gọi lá từ trái sang phải là $l_1 \dots l_8$; $C_1 = (l_1+l_2)/2$, $C_2 = (l_3+l_4)/2$, $C_3 = (l_5+l_6)/2$.
Sau khi tính xong $B_1 = \max(C_1, C_2)$, nút A (MIN) có β = B_1. Khi duyệt B_2 (MAX) và tính xong C_3, nếu $C_3 > B_1$ thì $B_2 \ge C_3 > B_1$ nên B_2 không thể làm giảm min của A → cắt toàn bộ C_4 (gồm l_7 và l_8).
Ví dụ: $l_1 \dots l_6$ = 2, 2, 0, 0, 6, 4 ($B_1 = 2$, $C_3 = 5 > 2$) → l_7 (và l_8) có thể mang giá trị bất kỳ.

b) Trường hợp 2 (A MIN → C ngẫu nhiên → B MAX → lá; nút viền đậm là lá thứ 8).

Có thể. $C_1 = (B_1+B_2)/2$ đã biết; A có β = C_1. Với $C_2 = (B_3 + B_4)/2$, B_3 đã biết và $B_4 = \max(l_7, l_8) \ge l_7$. Nếu $(B_3 + l_7)/2 > C_1$, tức $l_7 > 2C_1 - B_3$, thì $C_2 \ge (B_3 + l_7)/2 > C_1$ bất kể l_8 → A = C_1 → cắt l_8.
Ví dụ: $B_1 = B_2 = 0$ ($C_1 = 0$), $B_3 = 1$, $l_7 = 5$: $(1+5)/2 = 3 > 0$ → cắt l_8.

c) Trường hợp 3 (nút viền đậm là lá thứ 6, con phải của B_3).

Không thể. Để cắt l_6 phải chắc chắn $C_2 = (B_3 + B_4)/2$ không còn ảnh hưởng tới A, tức cần chặn được C_2 từ một phía. Tại thời điểm xét l_6, B_4 chưa được thăm và nằm trong một nhánh con khác nên giá trị của nó có thể là bất kỳ số hữu hạn nào (không có cận). Vì vậy dù B_3 lớn đến đâu, C_2 vẫn có thể rất nhỏ (nếu B_4 rất âm) hoặc rất lớn; ta không có cận nào cho C_2 → không thể kết luận bỏ qua l_6.

d) Vì sao cắt tỉa trong cây có nút ngẫu nhiên khác Alpha-Beta của Minimax?

Trong Minimax, giá trị của nút MAX/MIN chỉ phụ thuộc vào *một* con tốt nhất, nên chỉ cần biết một cận của một con (ví dụ $v \geq \beta$) là đủ để bỏ qua các con còn lại. Nút ngẫu nhiên lấy trung bình có trọng số của tất cả các con, nên giá trị của nó phụ thuộc vào mọi con: biết một con lớn hay nhỏ vẫn chưa nói được gì về trung bình nếu các con còn lại chưa biết hoặc không có cận. Do đó chỉ cắt được khi (i) các con còn lại đã được tính xong (như trường hợp 1 và 2, con bị cắt nằm sau), hoặc (ii) đã biết khoảng giá trị của lá (ví dụ lá $\in [-V, V]$) để chặn trên/dưới cho phần chưa thăm. Vì vậy việc cắt tỉa ở nút ngẫu nhiên khó hơn và kém hiệu quả hơn Minimax.

Bài 3. Câu hỏi đúng/sai

a) Luôn có $V_M \leq V_E$. ☑ Đúng ☐ Sai

Tại mỗi nút MIN, giá trị = $\min_i v_i$. Sau khi thay bằng nút ngẫu nhiên, giá trị = $\Sigma\, p_i v_i \geq \min_i v_i$ với *mọi* phân phối p (trung bình có trọng số không nhỏ hơn giá trị nhỏ nhất). Các nút MAX đơn điệu theo giá trị con nên bất đẳng thức được truyền từ dưới lên gốc: $V_E \geq V_M$.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☑ Đúng ☐ Sai

Chính sách Minimax đảm bảo ít nhất V_M khi đối thủ chơi tối ưu (trường hợp xấu nhất). Khi đối thủ được thay bằng ngẫu nhiên, kết quả tại mỗi nút ngẫu nhiên là một trung bình của các kết quả, mà mỗi kết quả ≥ giá trị MIN, nên kỳ vọng ≥ V_M (chứng minh quy nạp như ý a).

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☑ Sai

Chỉ chính sách *Expectimax* mới đạt V_E; chính sách Minimax chọn hành động tránh trường hợp xấu nhất nên có thể kém hơn.
Phản ví dụ: gốc MAX có hai hành động. Hành động x dẫn tới nút ngẫu nhiên với lá 4 (xác suất 0,1) và 100 (xác suất 0,9); hành động y dẫn tới nút ngẫu nhiên với hai lá 5 và 5. Minimax coi nút ngẫu nhiên là min: x → 4, y → 5 nên chọn y, lợi ích kỳ vọng 5 ($V_M = 5$). Expectimax: x → $0{,}1 \cdot 4 + 0{,}9 \cdot 100 = 90{,}4$ nên $V_E = 90{,}4 > 5$.

Trò chơi một người (kết quả hành động không xác định trước, thắng +1, thua -1): chọn các phương pháp phù hợp**.**

☑ Minimax với tiện ích trạng thái cuối +1/-1 ☑ Expectimax với tiện ích trạng thái cuối +1/-1
☑ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/-1 ở kết quả thắng/thua ☐ Không phương án nào

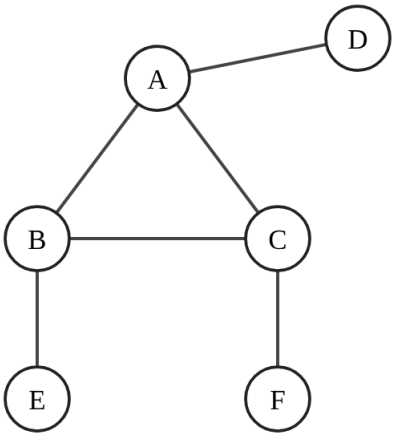
d) Giải thích lựa chọn và các giả định.

> Expectimax (phù hợp nhất): coi mỗi kết quả không xác định là nút ngẫu nhiên; cần biết phân phối xác suất của các kết quả. Giá trị tại gốc = kỳ vọng tiện ích = P(thắng) - P(thua) và chính sách tối ưu là cực đại hoá kỳ vọng này.
> Minimax (chỉ phù hợp có điều kiện): coi "môi trường" là đối thủ xấu nhất, chỉ cần biết tập kết quả có thể xảy ra (không cần xác suất). Nó cho chính sách bảo thủ nhằm đảm bảo kết quả trong trường hợp xấu nhất; hợp lý nếu muốn đảm bảo thắng hoặc không biết xác suất, nhưng không tối đa hoá kỳ vọng (thường thấp hơn Expectimax).
> Value Iteration (MDP): mô hình hoá bằng MDP với mô hình chuyển trạng thái biết trước, phần thưởng 0 ở trạng thái trung gian và ±1 ở trạng thái kết thúc, hệ số chiết khấu $\gamma = 1$ (hoặc trò chơi hữu hạn/kết thúc chắc chắn). Giá trị hội tụ $V^*(s)$ chính là kỳ vọng tiện ích nên cho cùng chính sách với Expectimax.

Giả định chung: một người chơi, trạng thái quan sát được đầy đủ, thắng/thua chỉ xác định ở trạng thái kết thúc.

Bài 4. CSP với Constraint Graph và lan truyền ràng buộc

a) Vẽ đồ thị ràng buộc.



Hình 3a. Đồ thị ràng buộc: 6 đỉnh A-F và 6 cạnh (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

b) Sau khi gán A, Forward Checking có thể thay đổi miền của biến nào?

Forward Checking chỉ lọc miền của các biến **chưa gán kề với A**: **B, C, D**.

c) Sau khi gán A (đã FC), tiếp tục gán B: FC tại B thay đổi miền của biến nào?

Các biến chưa gán kề với B: C và E (A đã gán; D không kề B).

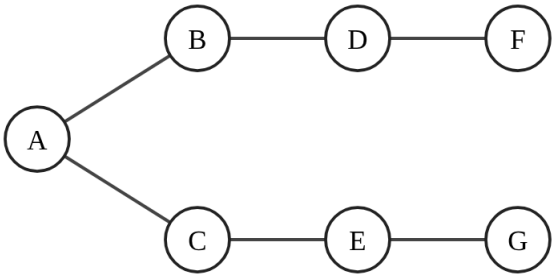
d) Sau khi gán A, duy trì Arc Consistency (MAC): miền của những biến nào có thể thay đổi?

Trước hết các biến kề A bị lọc: B, C, D. Khi miền của B thay đổi thì các cung (C,B), (E,B) phải được kiểm tra lại, nên E (và C) có thể đổi; khi miền của C thay đổi thì F có thể đổi (cung (F,C)). Vậy B, C, D, E, F đều có thể thay đổi (tất cả biến chưa gán).

e) Sau khi gán A và B (duy trì Arc Consistency): miền của những biến nào có thể thay đổi?

Gán B làm thay đổi trực tiếp các biến kề B chưa gán: C, E; thay đổi ở C lan sang F (cung (F,C)); thay đổi ở E không lan thêm vì B đã gán. D chỉ kề A (đã gán và đã được lọc ở bước trước) nên không đổi. Vậy C, E, F.

Thuật toán mới: chạy Arc Consistency (AC) ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, …); miền mỗi biến có kích thước d. Phân tích số lần backtrack trong trường hợp xấu nhất với đồ thị Hình 3.



Hình 3. Đồ thị ràng buộc: A-B, A-C, B-D, C-E, D-F, E-G. Đây là một **cây** (thực chất là đường đi F-D-B-A-C-E-G).

Cơ sở lập luận. (1) Trên đồ thị ràng buộc dạng cây, sau khi đạt arc consistency đầy đủ và không có miền rỗng thì mọi giá trị còn lại trong mọi miền đều thuộc một nghiệm. (2) Do đó nếu mỗi biến được gán sau biến "cha" của nó (thứ tự đi từ gốc xuống lá của cây) thì luôn tìm được giá trị tương thích, không bao giờ phải backtrack. (3) Backtrack chỉ xảy ra khi hai biến đã gán nằm ở hai nhánh khác nhau, nối với nhau bằng một đường đi qua các biến chưa gán: mỗi biến riêng lẻ vẫn mở rộng được thành nghiệm, nhưng cặp giá trị đã chọn có thể không cùng mở rộng được. Việc này chỉ bị phát hiện ở lần chạy AC kế tiếp (sau biến thứ chẵn) và gây ra backtrack.
Quy ước: mỗi lần AC phát hiện miền rỗng và phải hủy phép gán vừa rồi tính là 1 backtrack; giả sử bài toán có nghiệm.

f) Thứ tự A → B → C → D → E → F → G

Mỗi biến đều kề với một biến đã gán nằm phía gốc (B, C kề A; D kề B; E kề C; F kề D; G kề E), tức là thứ tự đi từ gốc A xuống lá. Hai biến cùng được gán trong một cặp (A,B), (C,D), (E,F) không bao giờ nối với nhau qua biến chưa gán, và AC ban đầu đã đảm bảo mọi giá trị chọn ra đều có giá trị tương thích ở biến kề. Vậy không bao giờ backtrack: 0 lần trong trường hợp xấu nhất.

g) Thứ tự G → E → C → A → B → D → F

Đây là thứ tự đi dọc đường G-E-C-A-B-D-F từ một đầu mút: mỗi biến kề đúng biến vừa gán trước đó, không có hai biến đã gán bị ngăn cách bởi biến chưa gán. Tương tự (f): 0 lần backtrack.

h) Thứ tự G → F → E → D → C → B → A

Hai biến đầu G và F nằm ở hai đầu mút của đường đi và bị ngăn cách bởi 5 biến chưa gán, tương tự cho các cặp sau: (G,F) rồi (E,D) rồi (C,B); biến cuối A được nối bởi cả hai phía (B và C). Các lần chạy AC sau biến thứ 2, 4, 6 có thể phát hiện cặp không tương thích:

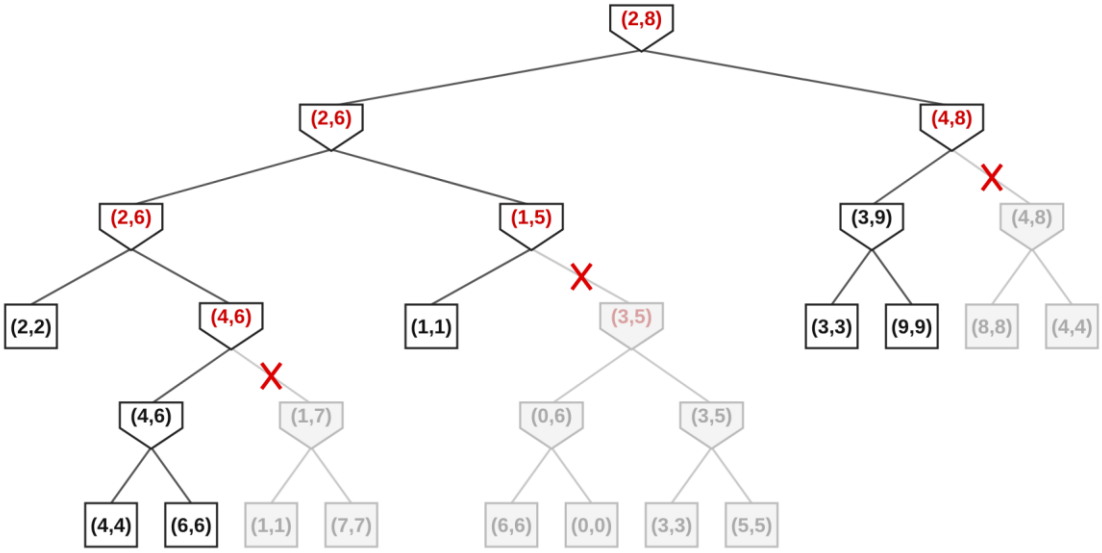
> Sau cặp (G, F): giá trị của F có thể không tương thích với G qua đường EcC-A-B-D; tối đa thử sai d - 1 giá trị của F trước khi tìm được giá trị đúng (mỗi giá trị của G đều thuộc một nghiệm nên ít nhất một giá trị của F đúng).
>
> Sau cặp (E, D): tương tự, tối đa d - 1 lần thử sai D (ngăn cách qua C-A-B).
>
> Sau cặp (C, B): tối đa d - 1 lần thử sai B (ngăn cách qua A).
>
> Gán A (biến thứ 7): AC lần trước đã đảm bảo miền của A không rỗng nên không còn backtrack.

Vậy trường hợp xấu nhất chỉ cần chặn trên $\leq 3(d - 1) = O(d)$ lần backtrack (không phải số mũ theo số biến). (Tôi cũng mô phỏng bằng chương trình trên các ràng buộc ngẫu nhiên với d = 2, 3, 4: thứ tự f và g luôn cho 0 backtrack, thứ tự h cho tối đa 1, 3, 5 lần - nằm dưới chặn 3(d - 1).)

Bài 5. Cây trò chơi Pacman và Casper

Nhận xét về cấu trúc: các nút ở độ sâu chẵn (kể cả gốc) là nút Casper - từ các cặp đã cho, (3,9) có lá (3,3), (9,9) nên u = min = 3, v = max = 9; ở độ sâu lẻ là nút Pacman. Quy tắc: nút Casper: u = min các u của con (Pacman tưởng Casper đối kháng), v = max các v của con (Casper thực sự thân thiện, chọn kết quả tốt nhất cho Pacman). Nút Pacman: chọn con có u lớn nhất (theo niềm tin), v = v của con được chọn.

a) Điền các cặp (u, v) còn thiếu. Giải thích sự khác biệt giữa u và v tại nút Casper.



Hình 4 (đã điền). Chữ đỏ là các giá trị được điền thêm; nhánh X/xám là nhánh có thể không cần duyệt khi tính u (câu b).

Nút	Tính toán	(u, v)
Pacman dưới LL (con (4,6), (1,7))	u = max(4, 1) = 4 → chọn (4,6) → v = 6	(4, 6)
Casper LL (con (2,2) và (4,6))	u = min(2, 4) = 2; v = max(2, 6) = 6	(2, 6)
Pacman dưới LR (con (0,6), (3,5))	u = max(0, 3) = 3 → chọn (3,5) → v = 5	(3, 5)
Casper LR (con (1,1) và (3,5))	u = min(1, 3) = 1; v = max(1, 5) = 5	(1, 5)
Pacman (con trái, con LL=(2,6), LR=(1,5))	u = max(2, 1) = 2 → chọn LL → v = 6	(2, 6)
Pacman (con phải, con (3,9), (4,8))	u = max(3, 4) = 4 → chọn (4,8) → v = 8	(4, 8)
Gốc Casper (con (2,6), (4,8))	u = min(2, 4) = 2; v = max(6, 8) = 8	(2, 8)

u khác v tại nút Casper: u là giá trị theo niềm tin của Pacman (Casper được coi là nút MIN, chọn kết quả tệ nhất cho Pacman); v là giá trị theo hành vi thực của Casper (nút MAX về lợi ích của Pacman, chọn kết quả tốt nhất). Vì thế tại mọi nút Casper $u \le v$ (ví dụ (3,9): u = 3, v = 9). Pacman ra quyết định theo u nhưng kết quả thực tế được quyết định bởi v.

b) Đánh dấu các nhánh có thể không cần duyệt (trái sang phải).

Alpha-Beta trên giá trị u (gốc Casper = MIN, các nút Pacman = MAX, xen kẽ), thứ tự trái sang phải:

> Casper LL: sau lá (2,2) có $\beta = 2$. Pacman dưới LL: nhánh (4,6) cho $u = 4 \ge \beta = 2$ → cắt nhánh (1,7) cùng hai lá (1,1), (7,7).
> Casper LR ($\alpha = 2$ từ nút Pacman cha): lá (1,1) cho $u = 1 \le \alpha = 2$ → cắt toàn bộ nhánh Pacman còn lại gồm (0,6), (3,5) và bốn lá (6,6), (0,0), (3,3), (5,5).
> Nút Pacman bên phải ($\beta = 2$ từ gốc): nhánh (3,9) cho $u = 3 \ge \beta = 2$ → cắt nhánh (4,8) cùng hai lá (8,8), (4,4).

Tổng cộng cắt 8 trong 14 lá; chỉ 6 lá được đánh giá: (2,2), (4,4), (6,6), (1,1), (3,3), (9,9). Giá trị u ở gốc vẫn là 2.
Lưu ý: phép cắt này đúng cho giá trị niềm tin u (trò chơi Minimax mà Pacman tưởng). Để biết cả giá trị thực v ở gốc (= 8, nhờ nhánh (4,8)) thì vẫn phải duyệt nhánh được Pacman chọn.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi thế nào?

Khi biết Casper thân thiện, Pacman mô hình mọi nút đều là nút MAX về lợi ích của mình, nên giá trị của cây là giá trị lớn nhất trong các lá = 9 (đường gốc → nhánh phải → nút (3,9) → lá (9,9)).
Giá trị Pacman *tin* ở gốc tăng từ u = 2 lên 9, và giá trị thực nhận được tăng từ v = 8 lên 9: ở nút Pacman

bên phải, Pacman bây giờ chọn (3,9) thay vì (4,8) (vì biết Casper sẽ chọn lá 9 chứ không phải 3), nên hiểu nhầm trước đó làm Pacman mất 1 đơn vị tiện ích.

Bài 6. Thuật toán tìm kiếm - Tính đầy đủ và tối ưu

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Hệ số nhánh b hữu hạn (và có nghiệm ở độ sâu hữu hạn). Đầy đủ cho cả tìm kiếm trên cây và trên đồ thị (đồ thị: không gian trạng thái hữu hạn hoặc b hữu hạn).	Tối ưu khi chi phí mọi bước bằng nhau (hoặc chi phí không giảm theo độ sâu); với chi phí bước khác nhau BFS tìm đường ít bước nhất chứ chưa chắc rẻ nhất.
Depth-First Search (DFS)	*Cây:* chỉ đầy đủ nếu cây hữu hạn, không có vòng lặp/nhánh vô hạn (có thể kẹt ở nhánh vô hạn hoặc lặp vô hạn). *Đồ thị* (có tập đã thăm): đầy đủ khi không gian trạng thái hữu hạn.	Không tối ưu: trả về nghiệm tìm được đầu tiên theo thứ tự duyệt, có thể rất sâu/đắt.
Iterative Deepening Search (IDS)	Giống BFS: đầy đủ khi b hữu hạn (kể cả không gian vô hạn), nhờ tăng dần giới hạn độ sâu.	Tối ưu khi chi phí mọi bước bằng nhau (như BFS). Với chi phí khác nhau cần dùng bản lặp theo chi phí.
A* Search	Đầy đủ khi b hữu hạn và chi phí mỗi bước $\geq \varepsilon > 0$ (để $f = g + h$ tăng không dừng ở giá trị hữu hạn); heuristic không âm.	*Cây:* tối ưu khi h chấp nhận được (admissible: $h(n) \leq h^*(n)$). *Đồ thị:* cần h nhất quán (consistent: $h(n) \leq c(n,n') + h(n')$), hoặc nếu chỉ admissible thì phải mở lại nút khi tìm được g nhỏ hơn.

Giải thích vì sao chi phí bước và heuristic ảnh hưởng. BFS/IDS chỉ đếm số bước nên sẽ sai khi các bước có chi phí khác nhau (đường ít bước có thể đắt hơn). Chi phí bước bằng 0 hoặc âm làm A*/UCS có thể lặp vô hạn trong chu trình chi phí 0 (mất tính đầy đủ) hoặc làm cận f mất ý nghĩa. Heuristic phải không đánh giá quá chi phí còn lại (admissible); nếu đánh giá quá, A* có thể bỏ qua đường tối ưu vì f của nó trông lớn hơn thực tế. Với tìm kiếm trên đồ thị, nếu h chỉ admissible mà không consistent thì một nút có thể bị đóng (đưa vào tập đã thăm) với g chưa tối ưu và các đường tốt hơn sau đó bị bỏ qua, nên cần consistent hoặc mở lại nút.

Bài 7. A* khi chi phí một cạnh thay đổi

Ký hiệu C* là chi phí tối ưu cũ, S → … → X → Y → … → G là đường tối ưu cũ. "Cập nhật" nghĩa là tận dụng g-giá trị, tập mở/đóng đã lưu thay vì chạy lại từ đầu (ý tưởng của các thuật toán tìm kiếm tăng cường như LPA*/D* Lite). Với trường hợp giảm chi phí, giả sử heuristic vẫn admissible/consistent với chi phí mới.

Tình huống	Chi phí tối ưu có đổi?	Cách cập nhật
a) Tăng n; X→Y thuộc đường tối ưu cũ, đã khám phá	Có thể đổi. Đường cũ tăng thêm n (C* + n); các đường khác không tăng nên một đường khác có chi phí trong khoảng [C*, C* + n) có thể trở thành tối ưu mới. Chi phí mới ∈ [C*, C* + n].	Cập nhật g của Y và mọi nút phía sau được thừa hưởng g qua cạnh này (+n), đưa các nút bị ảnh hưởng và các nút ở biên có $f \leq C^* + n$ trở lại tập mở rồi tiếp tục A* đến khi $f \geq$ chi phí tốt nhất. Không cần chạy lại từ đầu.
b) Giảm n; X→Y thuộc đường tối ưu cũ, đã khám phá	Có đổi nhưng đường không đổi. Chi phí mới = C* - n. Mọi đường khác giảm tối đa n (nếu cũng đi qua cạnh này) nên đường cũ vẫn tối ưu.	Chỉ cập nhật chi phí: đường giữ nguyên, trừ n khỏi tổng chi phí. Không cần tìm lại.
c) Tăng n; X→Y không thuộc đường tối ưu cũ nhưng đã khám phá	Không đổi. Đường tối ưu cũ không dùng cạnh này; các đường khác chỉ đắt thêm.	Không cần làm gì (có thể cập nhật g của các nút phụ thuộc cạnh này nhưng không ảnh hưởng nghiệm).
d) Giảm n; X→Y không thuộc đường tối ưu cũ nhưng đã khám phá	Có thể đổi. Đường đi qua X→Y có thể rẻ hơn C*: chi phí mới = min(C*, g(X) + (c - n) + h*(Y)).	Tính $g'(Y) = g(X) + c - n$; nếu nhỏ hơn g(Y) thì cập nhật và đưa Y trở lại tập mở, lan truyền g nhỏ hơn cho các nút phía sau, rồi tiếp tục A* đến khi $f \geq$ chi phí tốt nhất hiện tại. Không cần chạy lại từ đầu.

Tình huống	Chi phí tối ưu có đổi?	Cách cập nhật
e) Tăng n; X→Y không thuộc đường tối ưu cũ, chưa khám phá	Không đổi.	Không cần làm gì: cạnh chưa dùng tới và đắt hơn thì không thể giúp đường nào tốt hơn.
f) Giảm n; X→Y không thuộc đường tối ưu cũ, chưa khám phá	Không đổi nếu heuristic vẫn consistent: X chưa được mở rộng nên $f(X) \geq C^*$, khi đó f đi qua $Y = g(X) + (c - n) + h(Y) \geq g(X) + h(X) = f(X) \geq C^*$. (Nếu cạnh mới làm h mất tính consistent/admissible thì phải kiểm tra lại.)	Chỉ kiểm tra nhanh điều kiện trên; không cần mở rộng thêm. Nếu $g(X) + c - n + h(Y) < C^*$ thì đưa Y vào tập mở và tiếp tục A*.

Bài 8. Tác tử thông minh và môi trường

Ba kiểu tác tử: A (act = $f_A()$, bỏ qua percept: hành động tuỳ ý/ngẫu nhiên nhưng không phụ thuộc quan sát), B (phản xạ đơn giản: f_B(percept), chỉ phụ thuộc quan sát hiện tại) và C (f_C(toàn bộ lịch sử percepts)). Tác tử C có thể mô phỏng B và A (bỏ qua phần lịch sử).

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Các lớp có thể: B và C (C bao gồm B). Percept cho biết đầy đủ trạng thái, môi trường xác định nên hành động tối ưu là một hàm của trạng thái hiện tại (chính sách tối ưu tồn tại và chỉ phụ thuộc trạng thái hiện tại) - tác tử phản xạ B đã đủ. Tác tử A không phù hợp vì hành động tối ưu phụ thuộc trạng thái hiện tại, mà A bỏ qua percept.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Chỉ lớp C. Percept hiện tại không đủ để biết trạng thái; nhưng vì môi trường xác định và tĩnh, từ lịch sử percepts và hành động của chính tác tử có thể suy ra chính xác tập trạng thái có thể (belief state), hoặc cần nhớ mình đã đi/thử những gì (ví dụ mê cung chỉ nhìn được ô lân cận, hoặc phải biết đã thực hiện được bao nhiêu bước). B và A không nhớ nên không đủ cho mọi môi trường thuộc lớp này.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh

Chỉ lớp C. Cần theo dõi belief state (phân phối xác suất trên các trạng thái) tính từ toàn bộ lịch sử percepts; hành động tối ưu là hàm của belief state này. Tác tử B hay A (chỉ dùng percept hiện tại hoặc không dùng percept) không thể tích luỹ thông tin, nên không hành động hợp lý hoàn hảo trong mọi môi trường của lớp này. Ngẫu nhiên hoá hành động của tác tử (f có thể ngẫu nhiên) cũng không thay thế được trí nhớ.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý

Đúng. Ví dụ môi trường mà phép đo hiệu năng không phụ thuộc vào hành động (mọi hành động đều cho cùng kết quả/tiện ích), hoặc môi trường chỉ có một hành động hợp lệ duy nhất: khi đó mọi tác tử, kể cả tác tử chọn hành động ngẫu nhiên (kiểu A), đều tối đa hoá được độ đo hiệu năng nên đều hợp lý.